SHM-EM: A forecast-aware event management framework for heterogeneous engineering monitoring

Ji'an Liao[a,b], Zifa Wang[a,b,*], Dengke Zhao[a,b], Jianming Wang[a,b], Zhaoyan Li[a,b], Siran Yang[a,b]

[a] Key Laboratory of Earthquake Engineering and Engineering Vibration, Institute of Engineering Mechanics, China Earthquake Administration, Harbin 150080, China

[b] Key Laboratory Earthquake Disaster Mitigation, Ministry of Emergency Management, Harbin 150080, China

[*] Corresponding author: Zifa Wang, Zifa@iem.ac.cn. Institute of Engineering Mechanics, China Earthquake Administration, No. 29 Xuefu Road, Harbin 150080, China.

Abstract

Engineering-monitoring software often separates observations, forecasting, and event response, leaving model inputs schema-bound and forecast decisions difficult to audit. SHM-EM is an open-source framework that treats persisted forecasts as contextualized inputs to engineering-event management. It contributes three mechanisms: a versioned engineering-semantic data-model contract; a synchronized Project Future State for multi-target forecasts; and a controlled forecast-to-event transition that separates audited Evaluate from gated Execute. Validation uses a de-identified excavation case, a synthetic bridge workflow fixture, a 15-case matrix comprising one positive control, 12 failure-path cases, and two input-availability controls, runtime measurements, and an event provenance trace. The reference workflow integrates six fixed-version point-forecast bundles, 124 targets, 40 future steps, and 4,960 prediction rows. Docker/Linux reproduced the logical workflow, but not the exact Windows output hash; no tolerance was applied. The contribution is an auditable software workflow, not a forecasting algorithm or a claim of predictive generalization.

Keywords: engineering monitoring; structural health monitoring; time-series forecasting; event management; provenance; reproducible research software

Metadata

Nr	Code metadata description	Metadata
C1	Current code version	v1.0.1
C2	Permanent link to code/repository used for this code version	https://github.com/lja666/SHM-EM/releases/tag/v1.0.1 (fixed commit: d7cba1419145e6c75fe69ad63172af5f5abe5028)
C3	Legal code license	MIT License
C4	Code versioning system	Git
C5	Languages, tools, and services	Java 8; SQL; TypeScript; Python 3.10; Vue 3; Spring Boot 2.6.13; MyBatis 2.2.2; MySQL 8.4; PyTorch 2.11.0; Vite; ECharts; PowerShell 7; Docker Compose; OpenAPI
C6	Compilation requirements, operating environments, and dependencies	Back end: Java 8 and Maven 3.8+; database: MySQL 8.0+; front end: Node.js 20+ and npm; forecasting runtime: Python 3.10 with locked dependencies. Windows 10/11 with PowerShell 7 is the exact-output reference. The exercised Docker Compose Linux path reproduces component checks and the logical six-model-to-provenance workflow, but not a bitwise-identical normalized prediction-output hash.

Nr	Code metadata description	Metadata
C7	Developer documentation/manual	https://github.com/lja666/SHM-EM/tree/v1.0.1/docs (README, installation, reproduction, models, database, API, security, and data-availability documentation)
C8	Support email	nlfdzlja@163.com

1. Motivation and significance

Engineering monitoring combines heterogeneous measurements [1-3] and increasingly uses machine learning [4-6], yet applications and model scripts often retain project-specific tables, units, identifiers, and implicit feature order. Standards such as OGC SensorThings organize sensor observations [7,8], while generic CEP provides stream and event patterns [9]. GMFAgent addresses ground-motion-field estimation [10], and Predictive-SHM provides ingestion, model registration, forecasting, visualization, and alerts [11]. SHM-EM instead formalizes the downstream boundary through which persisted forecasts enter auditable engineering-event workflows. It neither replaces these systems nor claims SensorThings conformance. Table 1 compares documented responsibilities; *Not reported* means only that the cited primary source does not explicitly document the capability.

Table 1. Source-grounded responsibility comparison.

Capability	OGC SensorThings	Generic CEP	Predictive-SHM	SHM-EM
Observation access and semantics	Yes	Partial	Yes	Yes
Model-specific ordered input contract	Not applicable	Not applicable	Partial	Yes
Pluggable forecasting/model adapter	Not applicable	Not applicable	Yes	Yes
Shared prediction origin and future timeline	Not applicable	Not applicable	Not reported	Yes
Project-level future-state aggregation	Not applicable	Not applicable	Not reported	Yes
Rule/event evaluation	Not applicable	Yes	Partial	Yes
Execution-time eligibility recheck	Not applicable	Not reported	Not reported	Yes
Formal event-to-prediction provenance link	Not applicable	Not reported	Not reported	Yes

Forecasting methods are reviewed elsewhere [12-19]; SHM-EM registers and governs fixed-version models rather than proposing one. Model cards, data documentation, FAIR software practice, citation, and provenance inform its evidence design [20-25]. Its contributions are limited to:

1. **A versioned engineering-semantic data-model contract** binding observations, ordered features, targets, units, transformations, model artifacts, temporal settings, and hashes.

2. **A synchronized Project Future State** summarizing target, station, and project forecast risk on one timeline while separating observed and forecast risk.
3. **A controlled forecast-to-event transition** in which Evaluate retains an audit run without formal business records, while Execute reloads forecasts and rechecks eligibility, integrity, and rule semantics before creating events and provenance.

Here, *forecast-aware* means that persisted forecasts are aligned, inspected, and conditionally admitted to a formal workflow rather than used only for visualization.

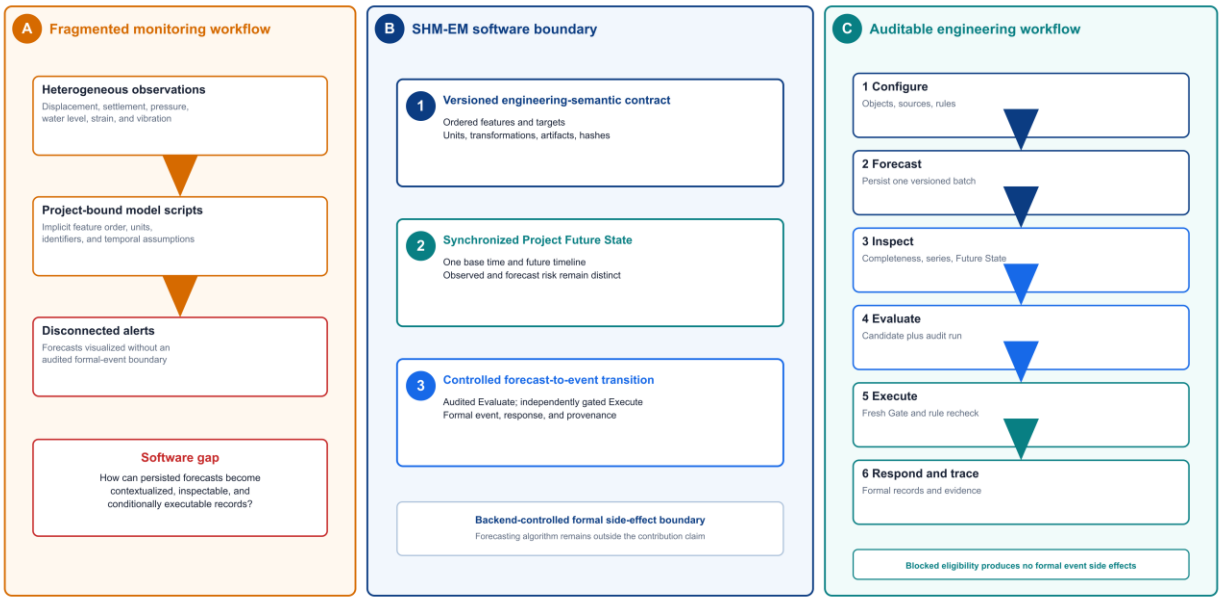

Fig. 1. Research gaps, the SHM-EM software boundary, and the forecast-aware user workflow.

1.1 Intended users and experimental setup

Researchers, analysts, and response personnel configure projects, stations, instruments, metrics, and approved observation mappings. PIT_PRE persists a prediction batch; users inspect its completeness, Project Future State, and joint series before Evaluate or Execute. Database scripts, APIs, tests, PowerShell, and Docker Compose reproduce the workflow; saved front-end state is not authoritative.

2. Software description

2.1 Software architecture

Fig. 2 shows four layers: Vue task views; Spring Boot services for observations, conversion, joint series, prediction Gate, Project Future State, rules, events, responses, and provenance; PIT_PRE for contract-driven Python inference; and MySQL persistence. This separates modelling dependencies from Java decision services, decouples inference from page access, and keeps event creation backend-controlled. MySQL is the validated implementation; registry and service interfaces define an extension boundary, but no alternative database adapter is validated.

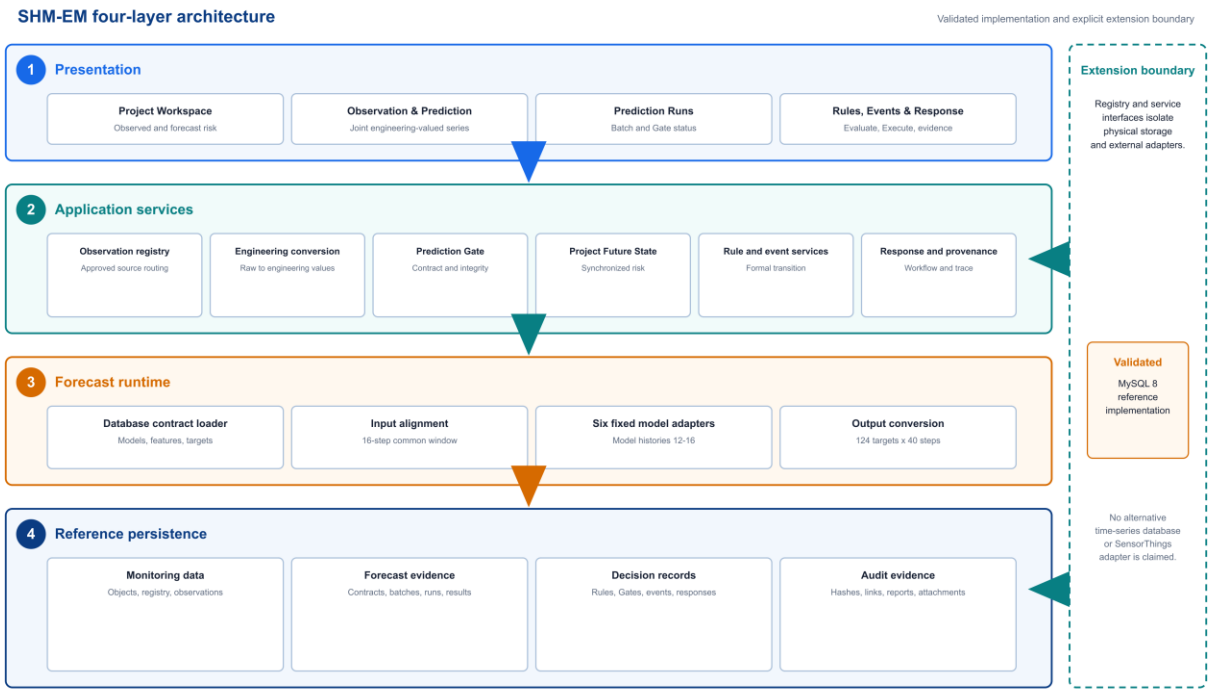

Fig. 2. Four-layer SHM-EM architecture. MySQL is the validated reference persistence implementation; the observation registry and service interfaces define the storage-adapter extension boundary.

2.1.1 Engineering monitoring object model and observation registry

A stable project-station-instrument-metric model links observations, forecasts, rules, and events:

```
project -> station -> instrument -> metric -> timestamped observation
```

Observations retain source, time, raw and engineering values, unit, quality, and conversion provenance. Clients request registry and metric codes, not table names. Four typed `em_obs_*` adapters and two registered acceleration partitions are supported; each source must satisfy allowlisted schema, time, unit, and conversion requirements.

2.1.2 Engineering-semantic data-model contract

The database-authoritative contract binds objects, ordered training features, targets, units, transformations, temporal settings, artifacts, preprocessors, scripts, runtime environment, and SHA-256 values. The public contract has 164 ordered source features and 124 targets. Frozen preprocessors select 114 columns for YD, XD, Strain, Pressure, and Water, and 164 for Settlement. A 16-step common window serves model-specific 12-16-step histories; Pressure declares 13 runner rows and consumes the final 12 (`m=10`, `lag=2`). Table 2 distinguishes aligned input widths from mapping and output counts.

Table 2. Verified model-bundle and tensor contract.

Model	Engineering target	Required history	Aligned input features	Output targets
YD	Deep horizontal displacement Y (mm)	16	114	42
XD	Deep horizontal displacement X (mm)	12	114	42

Model	Engineering target	Required history	Aligned input features	Output targets
Strain	Earth-pressure strain (microstrain)	13	114	14
Pressure	Earth pressure (MPa)	13	114	14
Water	Groundwater elevation (m)	13	114	2
Settlement	Surface settlement (mm)	12	164	10

Full fields, mappings, hashes, and model parameters are repository evidence. Listing 1 shows the manuscript-facing schema subset.

Listing 1. Compact extract from the versioned contract.

```
{
  "contractVersion": "pit_pre_contract_v1",
  "timeline": {"steps": 40, "stepMinutes": 3, "sharedBaseTime": true},
  "model": {"code": "settlement", "history": 12, "inputs": 164, "outputs": 10},
  "feature": {"order": 115, "code": "dtu1_point1_settlement_value",
              "rawUnit": "mm", "engineeringUnit": "mm", "required": true},
  "inputSchemaSha256": "5c2f6f0f...672daa65f1"
}
```

The schema-validated example and database export are in `docs/revision/examples/` and `artifacts/revision/manuscript/`.

Missing and asynchronous observations

Each required feature is matched backward-asof to a 3-min grid within one cadence; unresolved partial gaps use the declared linear interpolation and boundary-fill policy. Runs record signed source-time offsets, fill counts, ratios, and gap summaries. If an entire required feature is unavailable, or a required value remains unresolved, inference is rejected. Freshness is checked separately: OPERATIONAL uses wall-clock age, whereas REPLAY uses scenario time.

2.1.3 Multi-target rolling forecasts and Project Future State

A batch identifies its project, base time, cadence, horizon, and required models. Runs retain contract, input, artifact, alignment, and integrity metadata; results retain target, step, time, raw and engineering values, unit, conversion, and quality. Project Future State deterministically summarizes synchronized predictions and rule-bound risk without replacing formal rules or implying uncertainty estimation or multi-physics learning.

Algorithm 1. Policy-bound Project Future State aggregation.

```
INPUT project, batch, horizon, mode, reference time
1 Validate the active policy and its canonical hash.
2 Resolve a successful project-owned batch and positive horizon.
3 Inspect Gate eligibility for the requested temporal mode.
4 Load engineering-valued predictions and unit-compatible rule levels.
5 Evaluate each target by step with independent consecutive streaks.
6 Assign the highest activated severity and activation time.
7 Derive forecast risk; merge it with open observed-event risk.
8 Aggregate target, station, and future-timeline summaries.
9 Hash canonical state and return policy, Gate, risks, and summaries.
```

Inclusive operators and `between` include equality; strict operators do not. A nonmatch resets its streak, and severity governs risk ordering. Gate status is reported but does not alter the summary or become a hidden Execute prerequisite.

2.1.4 Controlled transition from forecasts to engineering events

`MetricSeriesPoint` exposes common object, metric, time, engineering value, unit, quality, source, and provenance fields; predictions add batch, run, model, step, conversion, and integrity context. Contract integrity, rule semantics, Evaluate, and Execute remain distinct. Evaluate inspects a REPLAY Gate, returns candidates, and persists one evaluation/audit run, but no formal event, Gate record, response, notification, report, evidence, or prediction link. Execute reloads the canonical series, persists a fresh Gate result, revalidates the rule, and only then creates formal records (Fig. 3).

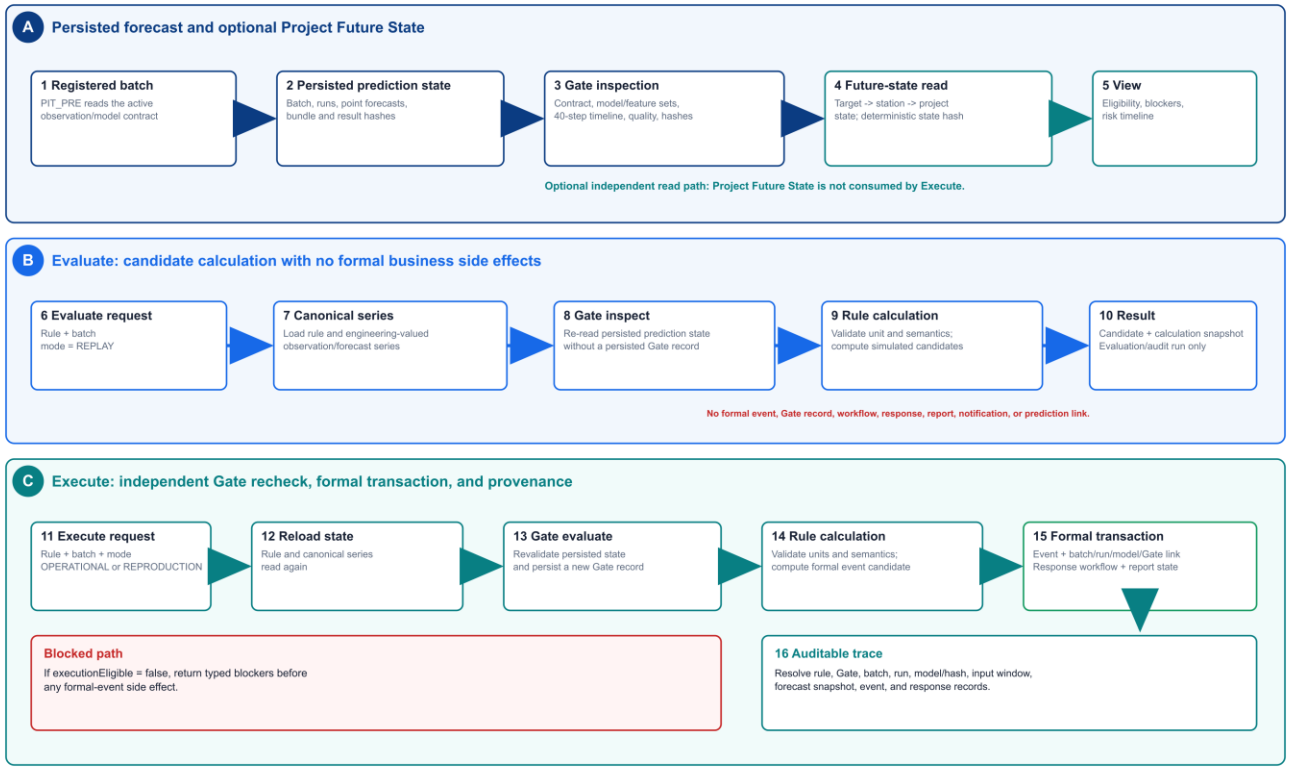

Fig. 3. Controlled sequence from persisted forecasts through optional Project Future State inspection, audited Evaluate, independently gated Execute, and formal provenance.

2.1.5 Response, provenance, and reproduction

Formal events may link responses, notifications, reports, generic media attachments, and audits; SHM-EM provides no camera control or capture subsystem. `em_event_prediction_link` records the batch/run, Gate, exceedance, lead time, peak, consecutive steps, snapshot, and result identity. Reproduction assets include the public window, fixed bundles, database scripts, locked dependencies, tests, entry points, and expected outputs [22-24,26-30].

2.2 Main functionalities

The interface supports project/observation registration, rolling predictions and Future State, Observation/Prediction rules with controlled execution, response/evidence management, and reproduction/audit. Fig. 4 combines the Project Workspace, joint series, and prediction-batch views; these panels are illustrative, not validation evidence.

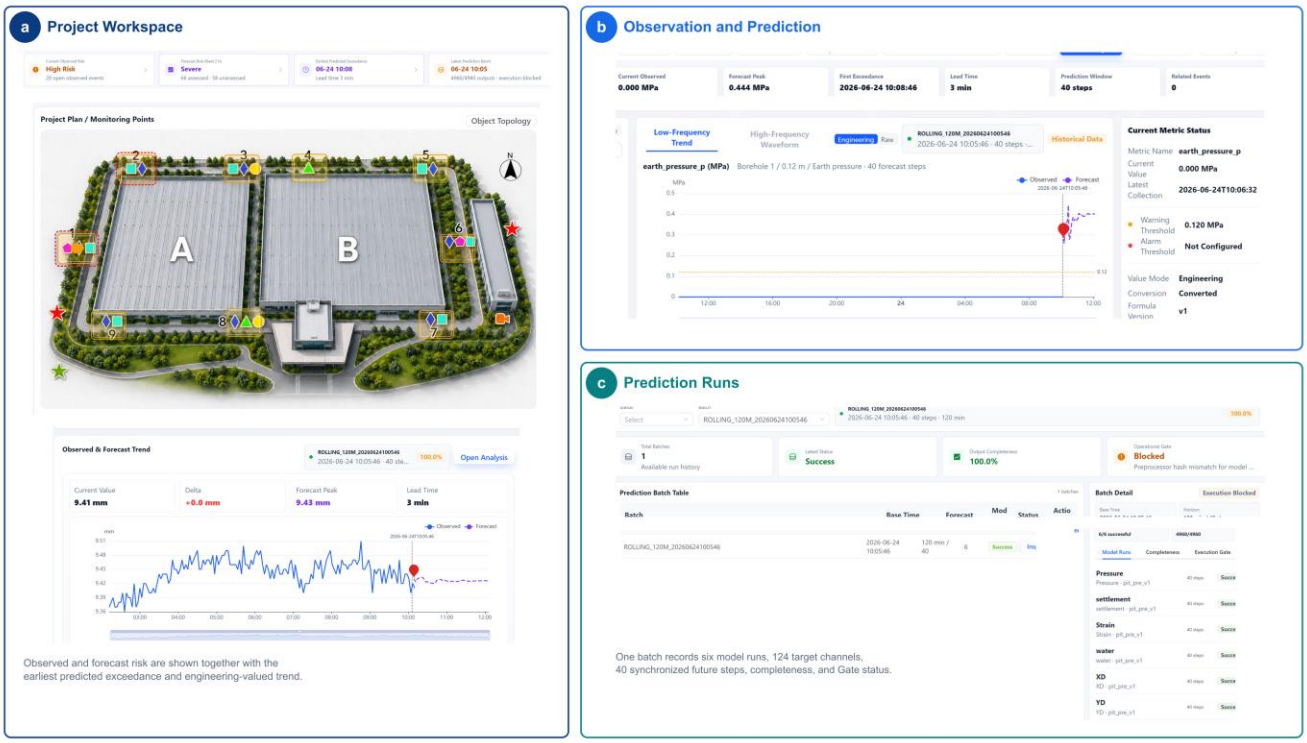

Fig. 4. Task-oriented interface views of SHM-EM: (a) project-level observed and forecast risk, (b) a joint engineering-valued observation/forecast series, and (c) prediction-batch completeness and execution eligibility. The interface is illustrative; quantitative validation is reported in the contract, failure-path, runtime, reuse, and provenance evidence.

2.3 Rule configuration and interfaces

Versioned rules define source (`OBSERVATION` or `PREDICTION`), metric, operator, threshold/unit, consecutive steps, severity, and temporal scope. The engine supports latest, extrema, mean, rate, absolute, interval, baseline-relative, and consecutive conditions for one metric. Cross-metric compound rules are outside v1.0.1. OpenAPI documents Future State, Evaluate, Execute, and provenance endpoints.

3. Software validation

Independent evidence families are reported separately because their behavior overlaps.

Table 3. Software testing evidence.

Test family	Cases/checks	Passed	Status
Backend unit/service/API tests	55	55	PASS
PIT_PRE contract/alignment/integrity tests	13	13	PASS
Validation matrix (P00, F01-F12, I01-I02)	15	15	PASS
Second-configuration end-to-end acceptance	7	7	PASS
Front-end typecheck and production build	2	2	PASS

Test family	Cases/checks	Passed	Status
Public reference end-to-end reproduction	1	1	PASS

No coverage percentage is claimed because no stable coverage instrument is included.

3.1 Public excavation-monitoring reference case

`SHM_EM_PUBLIC_SAMPLE` provides 2,464 de-identified, time-shifted observations from nine field points for displacement, pressure, groundwater, and hydrostatic level. Coordinates, operational identifiers, location, and events are removed. Sixteen 3-min steps support the 12-16-step model histories. The sample reproduces inference and software behavior, not independent accuracy or generalization. Its reference batch completed six models, 124 targets, 40 steps, and 4,960 rows through conversion, Gate, Future State, Evaluate, Execute, response, and provenance.

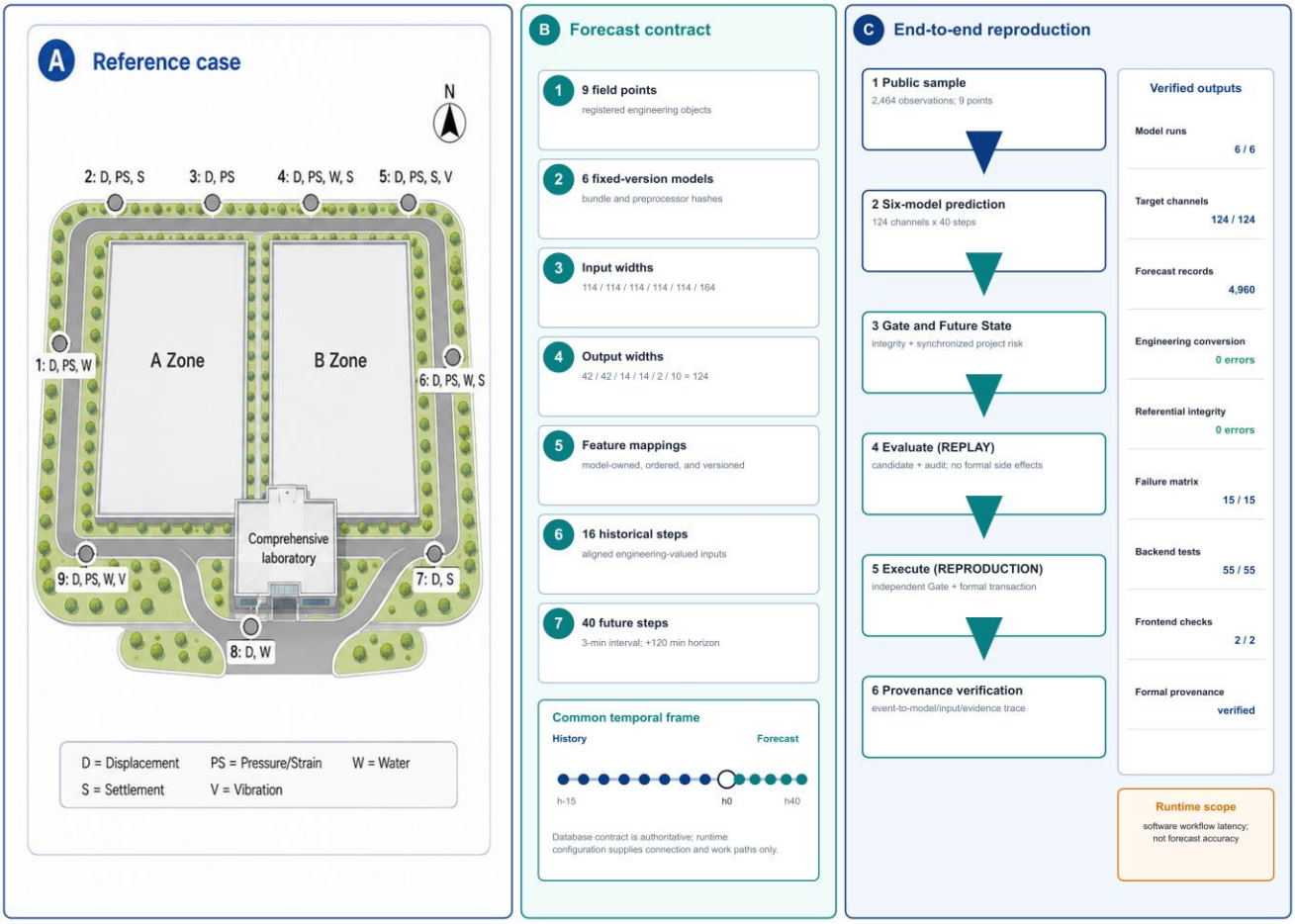

Fig. 5. Public reference case, verified six-model contract, common temporal frame, and end-to-end reproduction checks.

3.2 Cross-configuration reuse

One synthetic bridge configuration, used solely as a software-workflow fixture, tested registration and the end-to-end software boundary; it is not a bridge forecasting study.

Table 4. Synthetic bridge software-reuse fixture.

Registered or produced item	Count/result
Stations	3
Instruments	12
Compatible model bundles	2
Persisted forecast rows	1,120
End-to-end functional checks	7/7 PASS
Frozen core/schema modifications	0

The fixture produced 1,120 rows; integrity, Gate, Future State, Evaluate, Execute, response, provenance, API, and interface checks passed, while an unavailable mapping was rejected. Outputs are not interpreted as bridge-domain predictive validation or cross-domain accuracy. The result supports this tested configuration only.

3.3 Failure-path and execution-safety validation

A 15-case validation matrix comprising one positive control, 12 failure-path cases, and two input-availability controls ran in isolated databases. Blocked cases produced no formal event, response, report, evidence, or prediction link.

Table 5. Validation matrix and side-effect boundary.

Group	Cases	Condition	Expected/observed boundary
Positive control	P00	Valid reference prediction and rule	Execute succeeds; one formal event is created
Contract/model/batch	F01, F03, F04, F06, F08, F11	Incomplete steps; artifact hash mismatch; missing target; schema mismatch; failed run; failed batch	Execution Gate blocks
Timeline/quality/integrity	F02, F07, F09, F10	Stale prediction; temporal misalignment; corrupted persisted values with stale hashes; invalid quality	Gate or persisted-result integrity blocks
Rule semantics	F05	Incompatible engineering unit	Rule validation blocks
Evaluate-to-Execute mutation	F12	Persisted state is changed after Evaluate	Execute reload/recheck blocks
Input availability	I01, I02	Partial dropout; entire required feature unavailable	Declared alignment policy resolves the partial gap; unavailable required feature rejects input assembly

F09 motivated canonical-row integrity recomputation; F12 confirms that Execute reloads and rechecks state after Evaluate. This validates the tested fail-closed boundaries, not absolute safety.

3.4 Runtime and bounded scalability

Table 6 reports median/p95 Windows reference measurements from the fixed implementation. These are single-process reproduction workloads, not production-throughput tests.

Table 6. Selected runtime and bounded-scaling evidence (ms).

Operation	Workload	n	Median	p95
Full prediction batch	6 models; 4,960 rows	30	16,778.359	18,729.326
Execution Gate	4,960 rows	30	343.129	407.100
Project Future State	Reference batch	30	472.342	574.761
Rule Evaluate	Reference batch	30	269.465	313.340
Rule Execute	Reference batch	10	317.238	336.361
Provenance trace	One event	30	2.578	20.692
Gate S1	4,960 rows	10	2,406.939	2,666.804
Gate S2	49,600 rows	10	3,603.382	3,843.174

S1 and S2 are bounded endpoints, not evidence of linear scaling. The 50,000-row Gate cap is an application safeguard, not a MySQL capacity limit; no alternative time-series backend was measured.

3.5 Provenance and reproducibility

Table 7 summarizes one formal event trace; the complete 40-step chain is machine-readable in the repository.

Table 7. Concrete forecast-event provenance trace.

Trace element	Captured value
Formal event/rule	FEVT-4-f61b7667dcc01721aa2a; PRED_GROUND_SETTLEMENT_WARNING v2
Batch/run/model	Batch 40; run 236; settlement pit_pre_v1
Input/model integrity	Input-schema and model-artifact SHA-256 retained
Exceedance/Gate	3-min lead; 9.43204345-mm peak; Gate 1 eligible
Formal records	Event, workflow, four steps, report, prediction link

Windows 10/11 with PowerShell 7 is the exact-output reference. Docker/Linux completed the logical six-model -> 4,960 results -> Gate -> Future State -> Evaluate -> Execute -> provenance path, with structural target/step and contract hashes matching. Its normalized output hash differed (`exactPredictionReproduction=false`); maximum absolute difference was 0.00285349, `toleranceApplied=false`, and the full row-wise comparison is retained. Native Ubuntu-host execution was not captured.

4. Impact

4.1 Reproducible and auditable forecast integration

The contract, deterministic public window, expected outputs, tests, failure matrix, runtime records, and provenance make a model run inspectable rather than an implicit script invocation. Evidence supports contract checking, workflow reproduction, and traceability for the tested configurations, not forecasting superiority, calibrated uncertainty, production throughput, or reliability improvement.

4.2 Cross-configuration reuse

The bridge fixture registered three stations, 12 instruments, and two compatible bundles, produced 1,120 rows, passed 7/7 checks, and required zero frozen-core or schema modifications. This supports reuse for one fixture, not bridge accuracy, model transfer, arbitrary tables, or universal no-code onboarding.

4.3 Controlled event transition and traceability

Evaluate and Execute share engineering-valued rule calculations but differ at the formal side-effect boundary. The failure matrix shows blocked execution without formal records; the successful trace resolves event, rule, batch, model, input, Gate, forecast, and response. SHA-256 detects accidental or uncoordinated mutation, not a privileged attacker changing both data and hashes.

4.4 Current deployment and scientific scope

The six bundles emit point forecasts; Gate status is data/artifact/timeline/quality/freshness eligibility, not uncertainty or probabilistic risk. MySQL 8 is the only validated backend, the 50,000-row cap is application-level, and no SensorThings adapter or conformance result exists. The bridge fixture is not field validation, and arbitrary tables or models may require adapter work. Docker/Linux demonstrated logical, not bitwise, portability.

SHM-EM is a research reference without application authentication. Production use requires TLS, external identity, role-based and separate Execute authorization, least-privilege database access, secret management, protected audit storage, network controls, rate limits, and tested backup/restore. Forecasts require engineering review and must not be the sole basis for automated safety decisions.

5. Conclusions

SHM-EM contributes a versioned engineering-semantic data-model contract, a synchronized Project Future State, and a controlled forecast-to-event transition. The public case, workflow fixture, failure matrix, runtime evidence, provenance trace, and bounded container result support auditable integration for tested configurations. They do not establish predictive generalization, calibrated uncertainty, production security, backend neutrality, or exact cross-platform numerical reproduction.

6. Data and software availability

SHM-EM v1.0.1 is archived at https://github.com/lja666/SHM-EM/releases/tag/v1.0.1, fixed release commit `d7cba1419145e6c75fe69ad63172af5f5abe5028`. `SHM-EM-v1.0.1.zip` has SHA-256 `ea0973b7c82e06c3c8910ec36fcf2c3d47765a87d11552337a86c69de41a7cef`. Code and six model bundles use MIT; the de-identified sample and conceptual plan use CC BY 4.0. The sample supports the reported workflow but not model generalization. Restricted historical field data, locations, operational identifiers, credentials, generated databases, and local state are excluded; the repository documents this boundary and provides tests and expected outputs.

Acknowledgements

The authors thank the members of the research group for their contributions to software development and manuscript preparation. This work was supported by the Scientific Research Fund of the Institute of Engineering Mechanics, China Earthquake Administration (No. 2025B07), the National Natural Science Foundation of China (Nos. 52378543 and 52378544), and the National Key Research and Development Program of China (No. 2023YFC3805203).

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During preparation and revision, the authors used OpenAI ChatGPT and Codex for manuscript organization, language editing, software-code review, test and documentation preparation, and consistency checking. The authors reviewed, edited, and validated all AI-assisted outputs. AI-assisted code changes were human-reviewed and checked by the regression and reproduction procedures reported here. The authors take full responsibility for the publication's content.

References

[1] Farrar CR, Worden K. An introduction to structural health monitoring. Philos Trans A Math Phys Eng Sci. 2007;365(1851):303-315. https://doi.org/10.1098/rsta.2006.1928.

[2] Lynch JP, Loh KJ. A summary review of wireless sensors and sensor networks for structural health monitoring. Shock Vib Dig. 2006;38(2):91-128. https://doi.org/10.1177/0583102406061499.

[3] Hassani S, Dackermann U. A systematic review of advanced sensor technologies for non-destructive testing and structural health monitoring. Sensors. 2023;23(4):2204. https://doi.org/10.3390/s23042204.

[4] Bao Y, Li H. Machine learning paradigm for structural health monitoring. Struct Health Monit. 2021;20(4):1353-1372. https://doi.org/10.1177/1475921720972416.

[5] Azimi M, Eslamlou AD, Pekcan G. Data-driven structural health monitoring and damage detection through deep learning: state-of-the-art review. Sensors. 2020;20(10):2778. https://doi.org/10.3390/s20102778.

[6] Gomez-Cabrera A, Escamilla-Ambrosio PJ. Review of machine-learning techniques applied to structural health monitoring systems for building and bridge structures. Appl Sci. 2022;12(21):10754. https://doi.org/10.3390/app122110754.

[7] Broering A, Echterhoff J, Jirka S, Simonis I, Everding T, Stasch C, et al. New generation Sensor Web Enablement. Sensors. 2011;11(3):2652-2699. https://doi.org/10.3390/s110302652.

[8] Liang S, Khalafbeigi T, van der Schaaf H, editors. OGC SensorThings API Part 1: Sensing Version 1.1. OGC Implementation Standard 18-088. Open Geospatial Consortium; 2021. https://docs.ogc.org/is/18-088/18-088.html [accessed 1 September 2026].

[9] Cugola G, Margara A. Processing flows of information: from data stream to complex event processing. ACM Comput Surv. 2012;44(3):15. https://doi.org/10.1145/2187671.2187677.

[10] Zhao D, Wang Z, Wang J, Liao J, Li Z, Yang S. GMFAgent: domain knowledge-driven agent for ground motion field estimation. SoftwareX. 2026;34:102673. https://doi.org/10.1016/j.softx.2026.102673.

[11] Yang S, Li M, Liao J, Wang J, Zhao D, Li Z, et al. Predictive-SHM: an open-source, extensible software toolkit for multi-sensor structural health monitoring and time-series prediction. SoftwareX. 2026;35:102732. https://doi.org/10.1016/j.softx.2026.102732.

[12] Lim B, Zohren S. Time-series forecasting with deep learning: a survey. Philos Trans A Math Phys Eng Sci. 2021;379(2194):20200209. https://doi.org/10.1098/rsta.2020.0209.

[13] Kong X, Chen Z, Liu W, Ning K, Zhang L, Marier SM, et al. Deep learning for time series forecasting: a survey. Int J Mach Learn Cybern. 2025;16:5079-5112. https://doi.org/10.1007/s13042-025-02560-w.

[14] Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, et al. Attention is all you need. Adv Neural Inf Process Syst. 2017;30:5998-6008.

[15] Oreshkin BN, Carpov D, Chapados N, Bengio Y. N-BEATS: neural basis expansion analysis for interpretable time series forecasting. In: International Conference on Learning Representations; 2020.

[16] Lim B, Arik SO, Loeff N, Pfister T. Temporal Fusion Transformers for interpretable multi-horizon time series forecasting. Int J Forecast. 2021;37(4):1748-1764. https://doi.org/10.1016/j.ijforecast.2021.03.012.

[17] Zhou H, Zhang S, Peng J, Zhang S, Li J, Xiong H, et al. Informer: beyond efficient Transformer for long sequence time-series forecasting. Proc AAAI Conf Artif Intell. 2021;35(12):11106-11115. https://doi.org/10.1609/aaai.v35i12.17325.

[18] Wu H, Xu J, Wang J, Long M. Autoformer: decomposition transformers with auto-correlation for long-term series forecasting. Adv Neural Inf Process Syst. 2021;34:22419-22430.

[19] Nie Y, Nguyen NH, Sinthong P, Kalagnanam J. A time series is worth 64 words: long-term forecasting with Transformers. In: International Conference on Learning Representations; 2023.

[20] Mitchell M, Wu S, Zaldivar A, Barnes P, Vasserman L, Hutchinson B, et al. Model cards for model reporting. In: Proceedings of the Conference on Fairness, Accountability, and Transparency; 2019. p. 220-229. https://doi.org/10.1145/3287560.3287596.

[21] Gebru T, Morgenstern J, Vecchione B, Vaughan JW, Wallach H, Daume H III, et al. Datasheets for datasets. Commun ACM. 2021;64(12):86-92. https://doi.org/10.1145/3458723.

[22] Wilkinson MD, Dumontier M, Aalbersberg IJ, Appleton G, Axton M, Baak A, et al. The FAIR Guiding Principles for scientific data management and stewardship. Sci Data. 2016;3:160018. https://doi.org/10.1038/sdata.2016.18.

[23] Barker M, Chue Hong NP, Katz DS, Lamprecht AL, Martinez-Ortiz C, Psomopoulos F, et al. Introducing the FAIR Principles for research software. Sci Data. 2022;9:622. https://doi.org/10.1038/s41597-022-01710-x.

[24] Smith AM, Katz DS, Niemeyer KE; FORCE11 Software Citation Working Group. Software citation principles. PeerJ Comput Sci. 2016;2:e86. https://doi.org/10.7717/peerj-cs.86.

[25] Moreau L, Missier P, editors. PROV-DM: The PROV Data Model. W3C Recommendation. World Wide Web Consortium; 2013. https://www.w3.org/TR/2013/REC-prov-dm-20130430/ [accessed 25 July 2026].

[26] Wilson G, Aruliah DA, Brown CT, Chue Hong NP, Davis M, Guy RT, et al. Best practices for scientific computing. PLoS Biol. 2014;12(1):e1001745. https://doi.org/10.1371/journal.pbio.1001745.

[27] Sandve GK, Nekrutenko A, Taylor J, Hovig E. Ten simple rules for reproducible computational research. PLoS Comput Biol. 2013;9(10):e1003285. https://doi.org/10.1371/journal.pcbi.1003285.

[28] Pineau J, Vincent-Lamarre P, Sinha K, Lariviere V, Beygelzimer A, d'Alche-Buc F, et al. Improving reproducibility in machine learning research: a report from the NeurIPS 2019 reproducibility program. J Mach Learn Res. 2021;22(164):1-20.

[29] Lamprecht AL, Garcia L, Kuzak M, Martinez C, Arcila R, Martin Del Pico E, et al. Towards FAIR principles for research software. Data Sci. 2020;3(1):37-59. https://doi.org/10.3233/DS-190026.

[30] Stodden V, McNutt M, Bailey DH, Deelman E, Gil Y, Hanson B, et al. Enhancing reproducibility for computational methods. Science. 2016;354(6317):1240-1241. https://doi.org/10.1126/science.aah6168.